BDA Mini Project Report

SmartReco - E-Commerce Product Recommendation System

Submitted in fulfilment of the requirements of the
BDA Mini Project Semester VII in degree of
Bachelor of Computer Engineering

BY

Vinit Vishwakarma (Roll no. 48)
Utkarsh Yadav (Roll no. 68)
Amit Yadav (Roll no. 54)

Guide

Ms. Praniti Patil

Department Of Computer Engineering
Shree L. R. Tiwari College Of Engineering
Kanakia Park, Mira Road (E), Thane - 401 107, Maharashtra.

2025-26

BDA Mini Project Report Approval

This project report entitled **"SmartReco - E-Commerce Product Recommendation System"** by Vinit Vishwakarma (Roll No. 48), Utkarsh Yadav (Roll No. 68), and Amit Yadav (Roll No. 54) is approved for the Mini Project Semester VII project work of B.E CMPN at Shree L. R. Tiwari College of Engineering, in the academic year 2025-2026.

Internal Examiner

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated, or falsified any idea, data, fact, or source in this submission. We understand that any violation of these principles may result in disciplinary action by the Institute.

Signature

Vinit Vishwakarma (Roll No. 48)

Signature

Utkarsh Yadav (Roll No. 68)

Signature

Amit Yadav (Roll No. 54)

Date:

Place:

Table of Contents

1. Introduction

2. System Architecture

3. Implementation

4. Results

5. Conclusion

6. References

1. Introduction

1.1 Background

E-commerce platforms contain large volumes of customer purchase transactions. A customer may buy thousands of different products over time, making it difficult to manually identify which products should be shown next. Product recommendation systems solve this problem by learning patterns from historical interactions and presenting relevant products to each customer.

Big Data Analytics provides a systematic way to ingest, clean, aggregate, model, and visualise these transactions. SmartReco applies this lifecycle to the UCI Online Retail dataset. It converts raw invoices into customer-product interactions, learns implicit preferences using Apache Spark MLlib ALS, generates top-10 recommendations, evaluates them against future purchases, and displays the results in a Streamlit dashboard.

1.2 Problem Statement

Retail transaction data usually arrives as a large workbook containing invoices, product codes, descriptions, quantities, prices, dates, and customer identifiers. A data analyst needs to remove invalid transactions, handle cancellations and duplicates, find useful customer-product relationships, and produce recommendations that business users can inspect. Spreadsheet-based analysis does not scale well and cannot naturally provide personalised ranked recommendations for thousands of customers.

SmartReco addresses this problem through a local Spark pipeline and dashboard that transform raw retail data into actionable product recommendations and co-purchase insights.

1. Introduction

1.3 Objectives

1. To clean and validate the Online Retail transaction workbook using a repeatable data pipeline.
2. To aggregate purchase history into customer-product implicit feedback strengths.
3. To train an implicit-feedback ALS recommendation model using Spark MLlib.
4. To produce ranked top-10 recommendations for all available customers.
5. To evaluate recommendations with Precision@K and Recall@K using a temporal holdout.
6. To identify frequently co-purchased products using support, confidence, and lift.
7. To present portfolio summaries, product analytics, and recommendations through Streamlit.

1.4 Scope and Limitations

The system works on structured retail transaction data with invoice number, stock code, product description, quantity, invoice date, unit price, customer ID, and country fields. Positive, non-cancelled transactions with valid identifiers and prices are retained. The model uses implicit feedback, so a recorded purchase is treated as positive evidence while an unobserved product is not treated as a confirmed negative. The project is a local prototype; recommendations are not connected to a live commerce checkout or user account system.

1. Introduction

1.5 Relevance to Big Data Analytics

The project reflects the 3 Vs of Big Data at prototype scale. Volume is represented by 541,909 source transaction rows. Variety comes from identifiers, text descriptions, dates, quantities, prices, and customer fields. Velocity is represented by a repeatable batch pipeline that cleans, aggregates, models, and writes outputs for dashboard consumption. The value comes from converting raw purchase history into ranked recommendations and interpretable association rules.

The analytics lifecycle followed is ingestion, validation, cleaning, feature engineering, model training, evaluation, association analysis, output persistence, and visual presentation. Spark is used for distributed-style transformations and MLlib modelling, while PyArrow Parquet writing supports reliable output generation on Windows.

1.6 Organisation of the Report

Section 2 describes the architecture, data flow, schema, and model. Section 3 explains implementation details and technologies. Section 4 presents generated results and dashboard outputs. Section 5 concludes the work and lists future scope. Section 6 provides references.

2. System Architecture

2.1 Architectural Overview

SmartReco follows a layered batch analytics architecture. The data layer reads the Online Retail workbook. The processing layer validates and cleans records with Spark DataFrames. The feature layer aggregates transactions and assigns stable integer IDs. The modelling layer trains implicit ALS and generates recommendations. The presentation layer reads persisted Parquet and JSON outputs in Streamlit.

Figure 2.1: Layered system architecture of SmartReco.

The separation between processing and presentation means that the dashboard never retrains the model. It reads stable generated outputs, which makes the user interface responsive and keeps model execution reproducible.

2.2 Data Layer

The loader uses an explicit schema and openpyxl to read the Excel workbook without requiring a Spark Excel connector. The original workbook is never overwritten. Data cleaning creates canonical lower-case columns such as invoice_no, stock_code, description, quantity, unit_price, customer_id, invoice_date, and amount.

2. System Architecture

2.3 Processing and Modelling Layers

The cleaning stage removes rows with missing identifiers, invalid dates, non-positive quantities or prices, cancellation invoices, returns, and duplicate records. The feature stage groups valid purchases by customer and stock code. It calculates total quantity, purchase frequency, last purchase timestamp, and a representative product description. The interaction strength is defined as log1p(total positive quantity), which reduces the effect of unusually large purchases.

Stable integer IDs are assigned to customers and products because Spark ALS expects numeric user and item columns. The temporal evaluation split holds out the final interaction time while training on earlier activity. Recommendations that the customer already purchased are removed through a left-anti join.

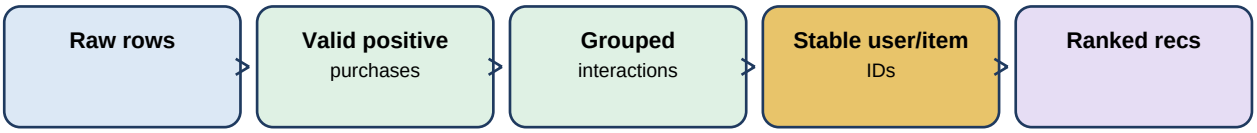

Figure 2.2: Transaction-to-recommendation data flow.

2.4 Presentation Layer

The Streamlit application provides an overview, product analytics, personalised recommendations, and evaluation and association views. It loads summary.json and data_quality.json for metrics, reads Parquet datasets for detailed views, and allows a customer to be selected before displaying the top-10 ranked products.

2. System Architecture

2.5 Dataset and Input Schema

Column	Meaning	Use
InvoiceNo	Invoice or transaction number	Cancellation detection and basket identity
StockCode	Product identifier	Item identity
Description	Product text description	Dashboard display
Quantity	Units in transaction line	Purchase strength and revenue
InvoiceDate	Transaction timestamp	Temporal split and analysis
UnitPrice	Unit price	Revenue calculation
CustomerID	Customer identifier	User identity
Country	Customer country	Source field retained for context

The inspected workbook contains 541,909 input rows. After validation and filtering, 392,692 positive purchases are retained. The generated interaction table contains 4,338 customers and 3,665 products.

2.6 Generated Outputs

Output	Purpose
cleaned_purchases	Validated transaction records
interactions	Aggregated customer-product strengths and metadata
all_customers	Top-ranked recommendations for customers
product_associations	Frequently co-purchased product pairs
summary.json	Portfolio counts, quantity, and revenue
evaluation.json	Precision, recall, and model information

2. System Architecture

2.7 Recommendation Model

SmartReco uses Spark MLlib ALS with implicitPrefs=True. The model learns latent customer and product representations from the interaction strength rather than from explicit star ratings. The configuration uses rank 10, eight iterations, regularisation parameter 0.1, alpha 1.0, top_k 10, and seed 42. A popularity fallback is available when the data is too sparse to train ALS.

For a user u and item i, ALS estimates a compatibility score from latent factors. The exact internal optimisation is handled by Spark MLlib. The application uses the resulting scores only for ranking, then removes products already purchased by the customer.

2.8 Association Analysis

Product associations are computed from basket-level co-occurrence. Support measures the fraction of baskets containing both products. Confidence measures the conditional probability of product B appearing when product A appears. Lift compares the observed co-occurrence with the probability expected under independence; a lift above 1 indicates a stronger-than-independent association.

3. Implementation

3.1 Project Structure

Module	Responsibility
main.py	Starts Spark and runs the pipeline
src/data_loader.py	Reads the workbook with an explicit schema
src/data_cleaning.py	Validates, types, filters, and reports quality
src/feature_engineering.py	Builds interactions and stable IDs
src/recommendation_model.py	ALS training and recommendation generation
src/association_rules.py	Product co-purchase metrics
src/evaluation.py	Ranking metrics
src/output_writer.py	Windows-friendly Parquet persistence
app.py	Streamlit dashboard

3.2 Data Cleaning

The cleaning module validates required source columns, casts fields to stable types, derives amount as quantity multiplied by unit price, marks cancellation invoices, filters invalid rows, removes duplicate records, and selects the canonical output schema. A data-quality report records input rows, duplicate rows, cancellations or returns, retained positive purchases, and dropped rows.

3. Implementation

Listing 3.1: Data cleaning and canonical fields

```
typed = (
    df.withColumn("invoice_no", F.trim(F.col("InvoiceNo").cast("string")))
      .withColumn("stock_code", F.trim(F.col("StockCode").cast("string")))
      .withColumn("description", F.trim(F.col("Description").cast("string")))
      .withColumn("quantity", F.col("Quantity").cast("double"))
      .withColumn("unit_price", F.col("UnitPrice").cast("double"))
      .withColumn("customer_id", F.col("CustomerID").cast("long"))
      .withColumn("invoice_date", F.to_timestamp("InvoiceDate"))
      .withColumn("amount", F.col("quantity") * F.col("unit_price"))
)
purchases = valid.filter(
    (F.col("quantity") > 0) & (F.col("unit_price") > 0)
).dropDuplicates()
```

The implementation keeps source validation close to ingestion so that downstream modelling receives a well-defined schema. This also makes data-quality counts reproducible across runs.

3. Implementation

Listing 3.2: Interaction feature engineering

```
interactions = (
    purchases.groupBy("customer_id", "stock_code")
    .agg(
        F.sum("quantity").alias("total_quantity"),
        F.countDistinct("invoice_no").alias("purchase_frequency"),
        F.max("invoice_date").alias("last_purchase"),
        F.first("description", ignorenulls=True).alias("description"),
    )
    .withColumn("strength", F.log1p("total_quantity"))
)
```

Stable IDs are then created with ordered row numbers over distinct customers and products. Keeping the original customer_id and stock_code alongside user and item makes the Parquet outputs useful for both modelling and dashboard display.

Listing 3.3: ALS recommendation configuration

```
ALS(
    userCol="user", itemCol="item", ratingCol="strength",
    rank=10, maxIter=8, regParam=0.1, alpha=1.0,
    implicitPrefs=True, coldStartStrategy="drop", seed=42
)
```

3. Implementation

3.3 Evaluation and Persistence

The pipeline uses a temporal holdout based on the maximum last-purchase timestamp. Earlier interactions form the training data, while the held-out future interaction is used to measure ranking performance. Precision@K counts recommended items that appear in observed future purchases divided by K. Recall@K counts observed future purchases recovered by the recommendations divided by the future items for that user.

Outputs are written as Parquet directories using PyArrow rather than requiring Hadoop's Windows-specific winutils.exe. JSON files store summary statistics, quality counts, and evaluation metrics. The Streamlit dashboard reads these persisted results and does not retrain the model.

3.4 Technologies Used

Layer	Technology	Role
Processing	Apache Spark	Distributed DataFrame transformations
Machine learning	Spark MLlib ALS	Implicit product recommendation
Data handling	pandas, PyArrow, openpyxl	Inspection, Parquet writing, workbook input
Presentation	Streamlit	Interactive dashboard
Language	Python 3	Pipeline and application implementation

4. Results

4.1 Data Quality and Portfolio Summary

The complete workbook contained 541,909 source rows. The cleaning pipeline detected 5,268 duplicate rows and 8,905 cancellation or return rows. It retained 392,692 positive purchase records for modelling and analysis.

Indicator	Value	Interpretation
Input rows	541,909	Rows in Online Retail workbook
Duplicate rows	5,268	Reported before deduplication
Cancellations or returns	8,905	Excluded from positive training data
Retained purchases	392,692	Valid positive purchase rows
Customers	4,338	Distinct customer IDs
Products	3,665	Distinct stock codes

Figure 4.1: Data-quality and scale summary generated by the pipeline.

The retained data represents 18,532 unique invoices, 5,152,002 positive units, and revenue of approximately 8,887,208.89 in the source currency. These values demonstrate that the application processes a substantial retail history rather than a toy single-customer example.

4. Results

4.2 Product Analytics

The dashboard presents best-selling products by total quantity and top products by revenue. These views are calculated from cleaned purchases grouped by stock code and description. A customer purchase-volume chart highlights customers with the highest aggregate spend.

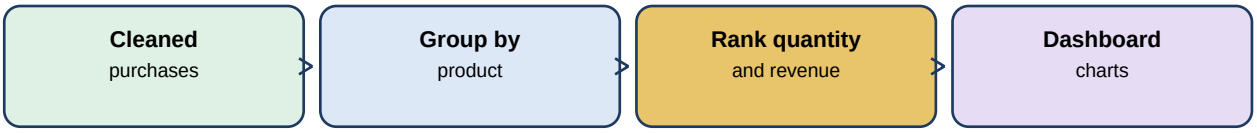

Figure 4.2: Product analytics flow.

The product views serve two purposes. First, they provide a business-level understanding of demand concentration. Second, they provide a baseline for judging personalised recommendations: a personalised model should identify relevant products without simply returning only the globally most popular items.

4.3 Personalised Recommendations

For each selected customer, the dashboard displays up to ten ranked products with rank, product code, description, and model score. Products already purchased by the customer are removed before display. The recommendations are read from data/recommendations/all_customers, so the dashboard remains a presentation layer over the generated model outputs.

4. Results

4.4 Model Evaluation

Metric	Value	Meaning
Algorithm	Implicit ALS	Spark MLlib recommendation model
Precision@10	0.00002359	Relevant observed future items per recommendation slot
Recall@10	0.07142857	Share of observed future items recovered
Evaluated users	4,239	Users included in temporal evaluation

The low Precision@10 and modest Recall@10 should be interpreted carefully. The evaluation counts only observed future purchases as positive evidence. An item that was recommended but not purchased is not necessarily a true negative because the customer may not have needed, seen, or had the opportunity to buy it. Sparse interactions and a strict temporal split also make the task difficult.

The metrics nevertheless provide a reproducible baseline for future model improvements, such as tuning rank and regularisation, adding recency weighting, filtering very rare products, or comparing ALS with popularity and item-item collaborative filtering.

4. Results

4.5 Association Rules

The association module identifies product pairs that occur in the same customer basket. The dashboard allows a product code to be selected and displays related pairs sorted by lift. Support, confidence, and lift together distinguish frequent products from genuinely informative co-purchase relationships.

Measure	Definition	Interpretation
Support	baskets containing A and B / all baskets	How common the pair is
Confidence	baskets containing A and B / baskets containing A	How often B follows A in a basket
Lift	confidence(A -> B) / support(B)	Strength above independent occurrence

Association results complement ALS recommendations. ALS models a customer-product preference across history, while associations describe products that appear together in baskets. A retail system can use both signals for personalised recommendations, cross-selling, and merchandising decisions.

4. Results

4.6 Dashboard Views

The SmartReco dashboard is organised into four working areas:

View	Displayed information
Overview	Transactions, customers, products, revenue, and data quality
Product analytics	Best-selling products, revenue leaders, and customer purchase volume
Personalised recommendations	Customer selector and ranked top-10 product recommendations
Evaluation and associations	Model metrics and frequently co-purchased products

The interface uses a wide layout so tables and charts remain readable. It also includes a fallback mode that shows live analytics from the workbook when generated Spark outputs are not yet available. This makes the application useful during initial setup while clearly indicating that the fallback does not train ALS or claim personalised recommendations.

Figure 4.3: Streamlit dashboard modules and their generated data sources.

4. Results

4.7 Discussion

The project completes the full BDA workflow from ingestion to presentation. The source workbook is cleaned without modifying the original file, aggregated into meaningful customer-product features, used to train an implicit recommendation model, evaluated with a temporal split, and persisted for dashboard use.

The most important practical result is the separation of model computation from user presentation. A new pipeline run can refresh Parquet and JSON outputs, while the Streamlit interface continues to use the same schema. This makes the prototype easier to test and extend.

The evaluation result also provides an honest baseline rather than an inflated claim. Since unobserved products are unknown rather than confirmed negatives, Precision@10 is very small and Recall@10 is 0.0714. The project should therefore be presented as a recommendation-system prototype and analytics demonstration, not as a production-grade commercial recommender.

4. Results

4.8 Reliability and Reproducibility

The pipeline is configured with a fixed random seed of 42 for ALS, explicit source and output paths, deterministic stable-ID ordering, and JSON summaries that can be inspected independently of the dashboard. The test suite verifies that interaction aggregation and stable IDs preserve distinct customers and products and calculate positive strengths.

Check	Result
Source workbook schema	Validated by the loader
Positive purchase filtering	392,692 retained rows
Stable customer and product IDs	Implemented and tested
Parquet outputs	Written through PyArrow
Recommendation display schema	rank, stock_code, description, score
Dashboard syntax	Validated with py_compile

These checks support repeatable execution on the current environment. For deployment, the project would additionally need automated integration tests, monitoring, dependency pinning, and a persistent serving layer.

4. Results

4.9 Limitations

The current system has several limitations. It uses only transaction history and does not include product categories, customer demographics, stock availability, seasonality, or item price changes. ALS quality depends on sufficient interaction history, and cold-start customers or products are difficult to recommend. The popularity fallback is useful for sparse cases but is not personalised.

The current evaluation is based on a single temporal split and observed future purchases. More robust evaluation would use multiple time windows, per-user minimum activity thresholds, ranking metrics such as MAP@K and NDCG@K, and business measures such as coverage and diversity. These limitations are appropriate for a semester prototype and define clear future work.

5. Conclusion

This mini project presented SmartReco, a local Big Data Analytics application for e-commerce product recommendations. The system reads the UCI Online Retail workbook, validates and cleans transactions, aggregates customer-product interactions, trains an implicit Spark ALS model, evaluates future-item recovery, computes product associations, and presents the outputs through Streamlit.

The pipeline processed 541,909 source rows and retained 392,692 positive purchase records from 4,338 customers and 3,665 products. It generated ranked recommendations and recorded a Recall@10 of 0.0714 for 4,239 evaluated users. The project therefore demonstrates how raw retail data can be transformed into an end-to-end recommendation workflow with reproducible outputs and an accessible dashboard.

The results should be understood as a prototype baseline. The application shows the complete analytics lifecycle and provides a foundation for more advanced recommendation experiments and eventual production deployment.

5. Conclusion

Future Scope

1. Improve model tuning by comparing ALS ranks, regularisation values, alpha, and top-k settings.
2. Add recency and frequency weighting so recent purchases influence recommendations more strongly.
3. Compare ALS with item-item collaborative filtering and popularity-based ranking.
4. Add product categories, country, seasonality, and price features for hybrid recommendations.
5. Evaluate with repeated temporal splits, MAP@K, NDCG@K, coverage, diversity, and novelty.
6. Add a database and authenticated API for multi-user deployment.
7. Scale the pipeline to larger retail streams using scheduled Spark jobs or structured streaming.
8. Add explainable recommendation reasons such as frequently bought together or similar customer behaviour.

SmartReco can consequently grow from a local academic prototype into a monitored recommendation service while preserving the current separation between data preparation, modelling, persisted outputs, and presentation.

6. References

[1] UCI Machine Learning Repository, Online Retail Data Set, University of California, Irvine.

[2] X. Meng et al., "MLlib: Machine Learning in Apache Spark," Journal of Machine Learning Research, vol. 17, no. 34, 2016.

[3] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," IEEE Computer, vol. 42, no. 8, pp. 30-37, 2009.

[4] R. M. Bell and Y. Koren, "Lessons from the Netflix Prize Challenge," SIGKDD Explorations, vol. 9, no. 2, 2007.

[5] Apache Spark Documentation, Spark MLlib Collaborative Filtering.

[6] pandas Documentation, DataFrame and Parquet interfaces.

[7] Streamlit Documentation, Data Apps in Python.

[8] PyArrow Documentation, Parquet file format and Python API.

[9] W. McKinney, *Python for Data Analysis*, O'Reilly Media.

[10] SmartReco project source code and generated outputs, BDA Mini Project repository, accessed October 2026.